

API TESTING MASTERCLASS

API Authentication Guide

API keys, Basic, Bearer/JWT and OAuth 2.0 — for testers.

Most real APIs are protected, so you cannot test them without understanding how authentication works. This guide covers the four schemes you will meet most often, how to set each one in Postman, and — crucially — how to test that access control actually holds.

Authentication vs Authorisation

These two words are used interchangeably in conversation but mean different things, and the difference shows up directly in status codes.

- Authentication — "who are you?". Proving identity with a credential (key, password, token). Failure returns 401 Unauthorized.
- Authorisation — "are you allowed to do this?". Checking whether an authenticated identity has permission for the action. Failure returns 403 Forbidden.
- Quick test heuristic: 401 means your credential is missing, malformed or expired; 403 means your credential is valid but you lack the rights.

The Four Common Schemes

Scheme	How it works	Where the credential goes	Tester notes
API key	A single secret string identifies the caller	Header (e.g. X-API-Key) or query parameter	Simple; often no per-user identity. Test missing/invalid key → 401
Basic	Base64 of username:password sent each request	Authorization: Basic <base64>	Not encrypted, only encoded — must be over HTTPS. Test wrong password → 401
Bearer / JWT	A signed token issued after login, sent on each call	Authorization: Bearer <token>	Token expires; test missing/expired/tampered token
OAuth 2.0	Exchange credentials/grant for a short-lived access token via an auth server	Authorization: Bearer <access token>	Multi-step flow; test token refresh and scope limits

Setting Each in Postman

Use the Authorization tab on the request or folder rather than hand-typing headers where possible — Postman builds the correct header for you.

1. API key: Authorization tab → type "API Key" → set the key name, value and whether it goes in Header or Query. Store the value as {{apiKey}}.
2. Basic: Authorization tab → type "Basic Auth" → enter username and password (use {{user}} / {{password}} variables).
3. Bearer/JWT: Authorization tab → type "Bearer Token" → paste or reference the token as {{authToken}} (typically captured from a login response).
4. OAuth 2.0: Authorization tab → type "OAuth 2.0" → configure the token URL, client id/secret and scope, then "Get New Access Token"; Postman stores and injects it.
5. Set auth at the collection or folder level so child requests inherit it, and override only where needed.

JWT Structure

A JWT is three Base64URL-encoded parts joined by dots: header.payload.signature. It is signed, not encrypted, so anyone can decode and read the header and payload — never put secrets in it.

- Header — the algorithm and token type, e.g. {"alg":"HS256","typ":"JWT"}.
- Payload — the claims: who the token is for (sub), when it expires (exp), and roles/scopes.
- Signature — proves the token has not been altered; the server verifies it with its secret/key.
- Expiry — the exp claim is a Unix timestamp. Decode it (jwt.io or a script) to see when the token dies; an expired token must be rejected with 401.

Testing Access Control

A protected endpoint that lets the wrong caller in is a serious security bug. Do not just confirm the happy path works — actively try to break in.

- No token — send the request with the credential removed; expect 401.
- Expired token — use a token past its exp; expect 401, not a silent success.
- Tampered token — change a character in the payload/signature; expect 401.
- Wrong-user token — authenticate as User A and try to read/modify User B's resource; expect 403 (or 404 by design), never User B's data.
- Insufficient scope/role — use a valid low-privilege token on an admin action; expect 403.

Common Mistakes

- Confusing 401 and 403, then raising bugs against the wrong behaviour.
- Only testing that a valid token works, never that missing/expired/wrong-user tokens are rejected.
- Pasting a token that has since expired and reporting a false "auth broken" bug.
- Committing real tokens or credentials into an exported collection.
- Assuming Basic auth is secure because it is "encrypted" — it is only Base64-encoded.

INSIDE STLC PRO TIP

The most valuable API auth bugs are broken access control: a valid token for User A reaching User B's data. Always test "authenticated as the wrong user", not just "not authenticated at all" — it is where real vulnerabilities hide.